

Automated Problem-to-Solution Generation for Tokamak Fusion-Plasma Simulations

A hierarchical multi-agent PETSc system: from a plain-language prompt to verified, real-machine Grad-Shafranov equilibria

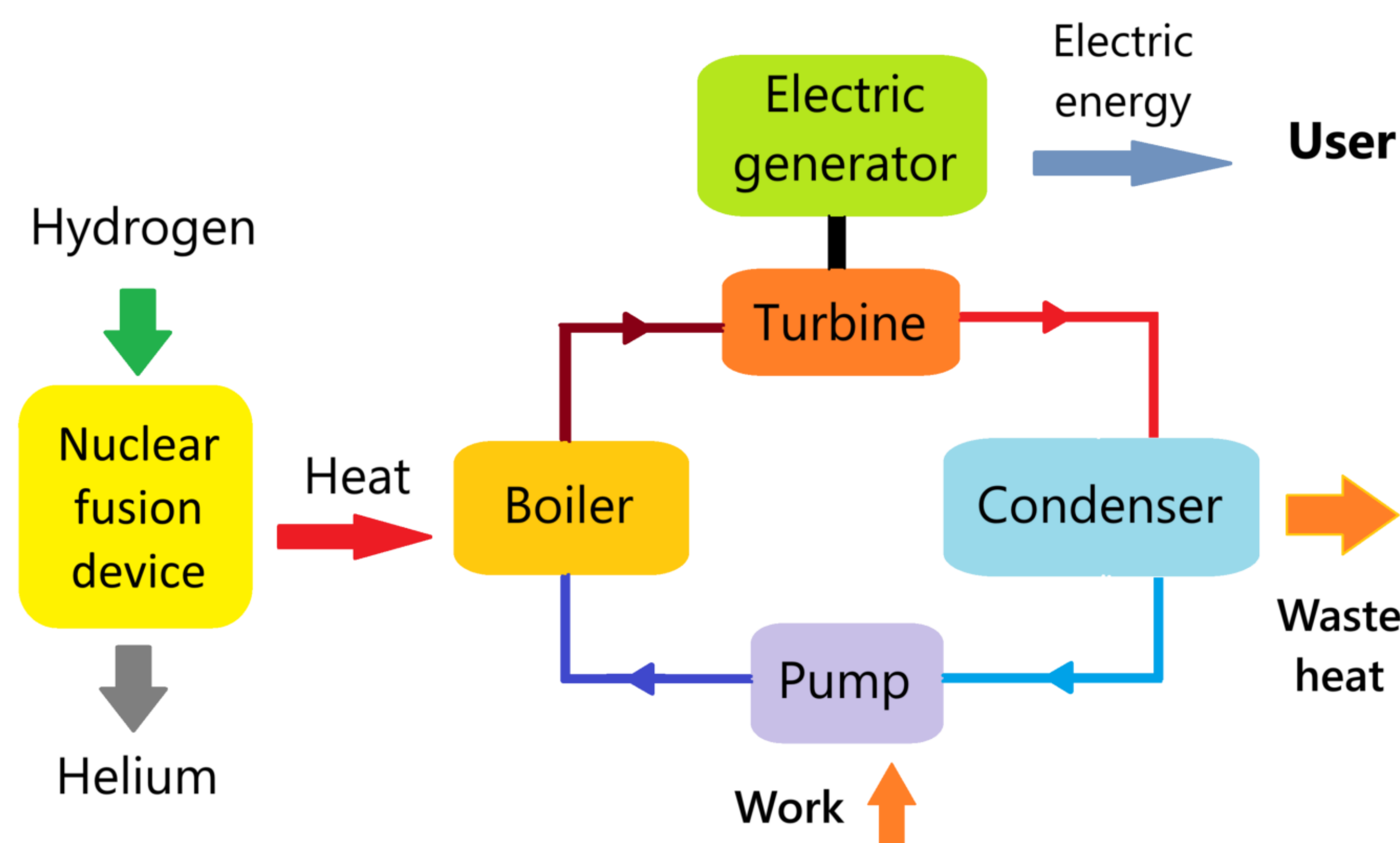
Sarthak Sharma (State University of New York at Buffalo) · Dr Junchao Zhang (Mathematics and Computer Science Division, Argonne National Laboratory) · US-RSE 2026

1 Motivation

Building correct, fast HPC simulation code demands scarce, implicit expertise. Can a multi-agent AI system automate the path from a plain-language idea to VERIFIED code for a real fusion problem?

We answer yes for the tokamak Grad-Shafranov equilibrium — with verification as a first-class deliverable.

Fusion energy



A tokamak turns fusion heat into electricity; its plasma equilibrium is governed by the Grad-Shafranov equation.

2 The problem

Axisymmetric ideal-MHD force balance for the poloidal flux $\psi(R,Z)$:

$$\Delta^* \psi = -\mu_0 R^2 p'(\psi) - F F'(\psi)$$

Sets the flux surfaces, magnetic axis, plasma shape, and safety factor q .

Solov'ev profiles admit an EXACT solution → a rigorous verification anchor.

3 The multi-agent system

Specialist agents (MCP servers) on ANL's Argo (Claude Opus 4.8):

- Modeling → PDE identity & weak form
- Numerical Analysis → grid, discretization, solver
- HPC Code Generation → writes, compiles & runs PETSc C
- Driver → records every artifact (full provenance)

4 The pipeline in action

Prompt: “the Grad-Shafranov equilibrium for a tokamak plasma.”

Modeling → ‘Grad-Shafranov equation’, steady.

Numerical Analysis → nonlinear → SNES on a DMDA.

Code Generation → ONE 252-line parametrized PETSc solver.

Compiled & ran on 1 and 4 MPI ranks — no human edits.

5 Generated solver (excerpt)

```
/* GS operator (normalized x=R/R0), 2nd-order FD */
DMDACreate2d(...,DMDA_STENCIL_STAR,...,&da);
dxx=(u[j][i+1]-2*u[j][i]+u[j][i-1])/hx2;
dyy=(u[j+1][i]-2*u[j][i]+u[j-1][i])/hy2;
f[j][i]=dxx-dx/x+dyy-((1-A)*x*x+A);
/* Dirichlet BC = analytic CF psi (nonzero) */
f[b]=u[b]-PsiExact(&u,x,y);
SNESetJacobian(snes,J,J,FormJacobian,&u);
SNESolve(snes,NULL,X);
```

6 Real-machine equilibria

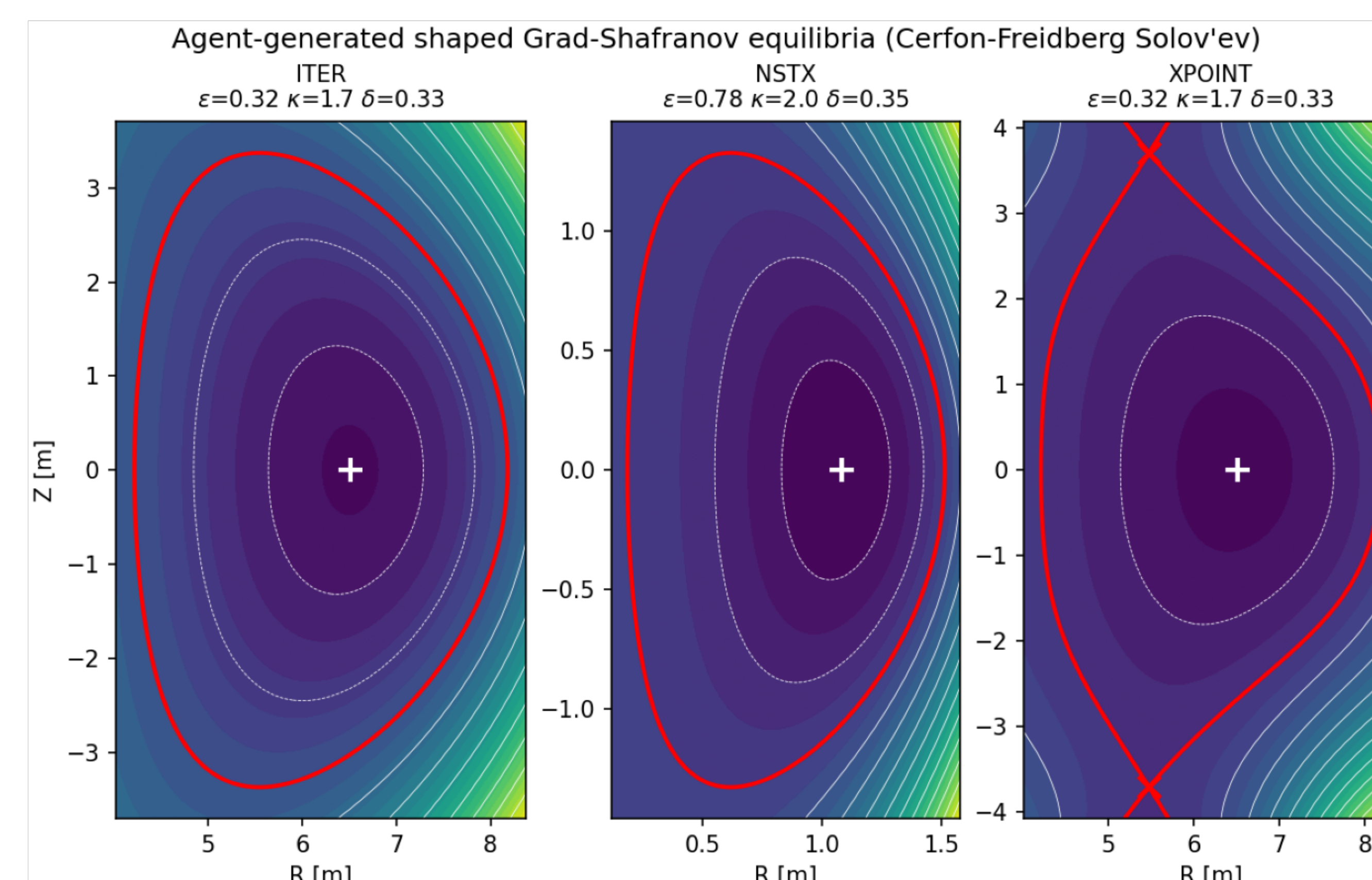


Fig. 1 One agent-generated solver → ITER, spherical-tokamak (NSTX-like) and diverted double-null (X-point) equilibria. Red = separatrix; × = X-points.

7 Verification: 2nd order

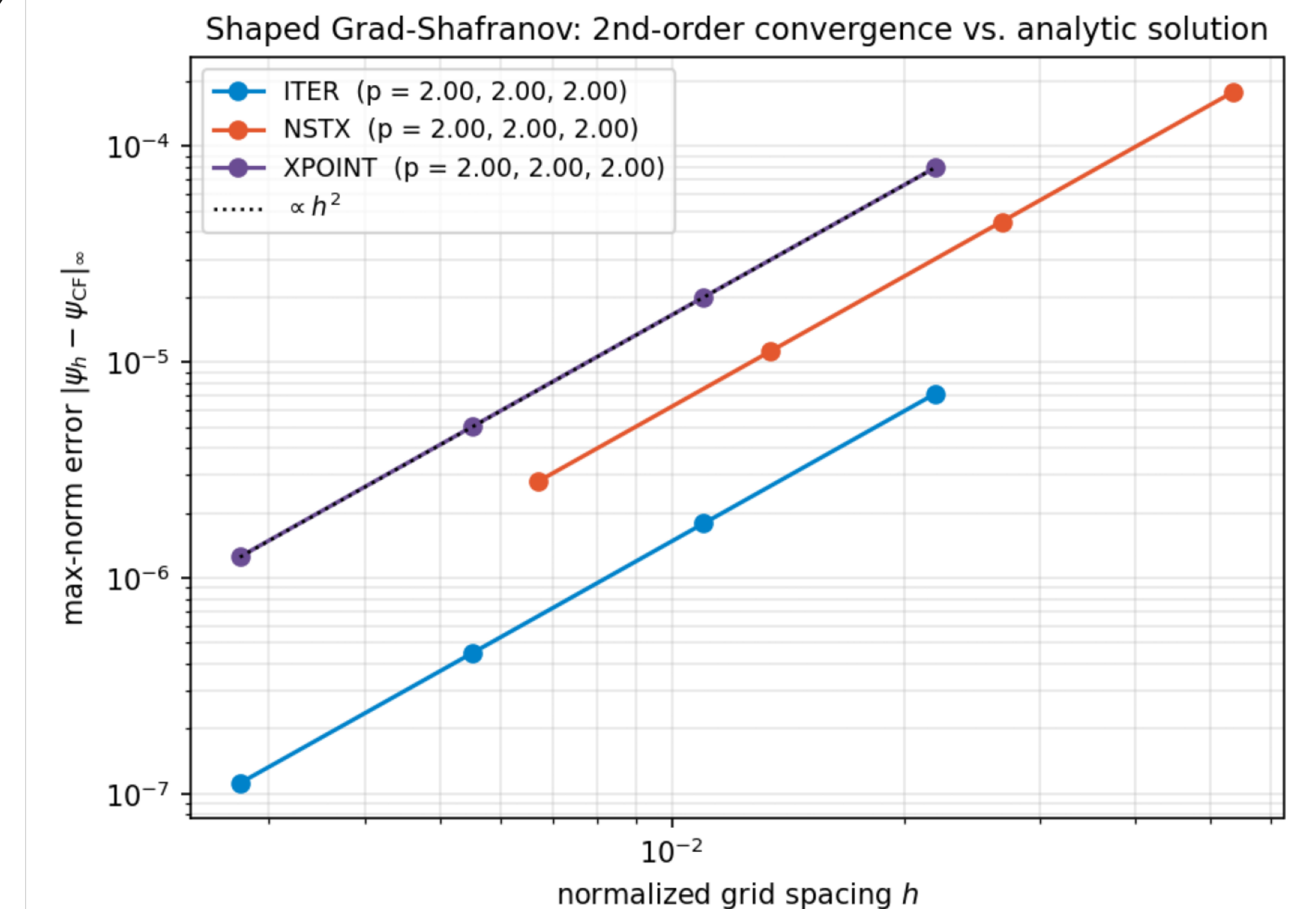


Fig. 2 Method of manufactured solutions vs the exact Cerfon-Freidberg solution: observed order $p = 2.00$ for all three machines.

8 Safety factor & FreeGS cross-check

Safety-factor $q(\psi_N)$ computed from the verified flux surfaces:

ITER $q_{95} \approx 2.9$ · NSTX $q_{95} \approx 12.4$ · X-point $q_{95} \approx 3.2$

Cross-checked vs FreeGS's independent q on the SAME field → agree to $< 0.2\%$.

Measured κ , δ match the input shaping to $\sim 1\text{--}2\%$.

9 Results & conclusions

One 252-line solver, 0 hand-written, verified on 3 real-machine equilibria.

2nd-order accurate ($p = 2.00$) on every case; runs on 1 & 4 MPI ranks.

Efficiency: ~ 311.4 s wall-clock; ~ 27 LLM completions.

Verification-driven: exact-solution convergence + independent q cross-check.

github.com/engineer-scientist/petsc_mcp_servers_tokamak